

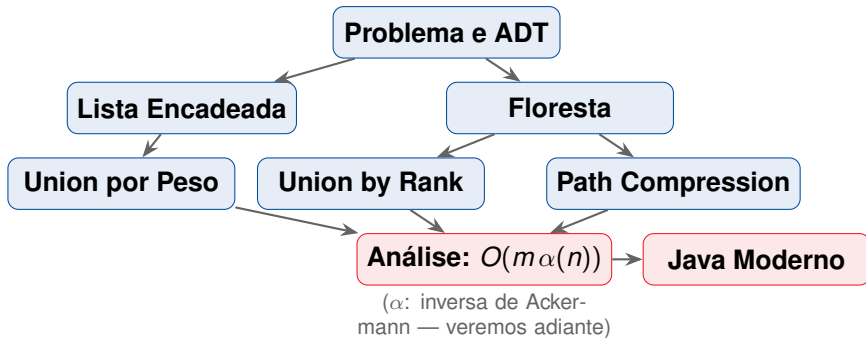
Conjuntos Disjuntos

Union-Find, Path Compression e Representação por Floresta

Prof. Andre Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

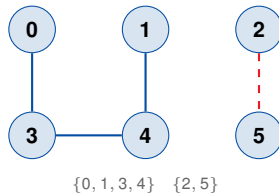


Motivação

Problema

Dados n objetos, processar duas operações:

- **Union**(p, q): registrar que p e q estão conectados
- **Connected**(p, q): p e q estão no mesmo componente?



Conectividade é transitiva: se $p-q$ e $q-r$,
então $p-r$.

Matematicamente: **relação de equivalência**.

Algoritmos de grafos

- Kruskal — MST em $O(m \alpha(n))$
- Componentes conexas on-line
- Detecção de ciclos
- Menor caminho com pesos (Borůvka)

Aplicações em sistemas

- Unificação de tipos (compiladores)
- Segmentação de imagens
- Percolação (Monte Carlo)
- Particionamento de memória (GC)

Três operações fundamentais

MAKE-SET(x) Cria um conjunto $\{x\}$; x é seu próprio representante.

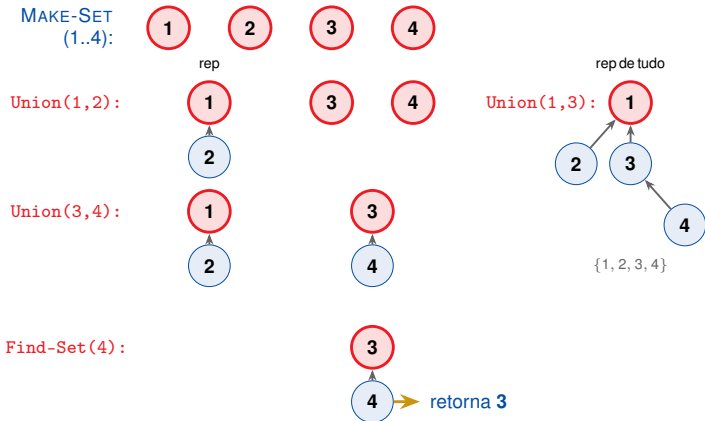
FIND-SET(x) Retorna o *representante* do conjunto que contém x .

UNION(x, y) Funde os conjuntos de x e y em um único conjunto.

Invariante fundamental

$\text{FIND-SET}(x) = \text{FIND-SET}(y) \iff x \text{ e } y \text{ pertencem ao mesmo conjunto.}$

Exemplo: as três operações em ação



1. **Fase de inicialização:** n chamadas a MAKE-SET — uma por objeto.
2. **Fase de processamento:** as $m - n$ operações restantes são UNION (no máximo $n - 1$) e FIND-SET.

Meta de desempenho

Processar m operações em tempo total $O(m\alpha(n))$,
onde α é a **inversa de Ackermann**: $\alpha(n) \leq 4$ para qualquer n concebível.

Na prática, é **praticamente linear**: $O(m\alpha(n))$ com $\alpha(n) \leq 4$.

Implementação	Find-Set	Union
Lista encadeada ingênua	$O(1)$	$O(n)$
Lista + union por peso	$O(1)$	amort. $O(\lg n)$
Floresta ingênua	$O(n)$	$O(1)$
Floresta + union por rank	$O(\lg n)$	$O(1)$
Floresta + path compression	amort. $O(\lg n)$	$O(1)$
Floresta + rank + PC	$O(\alpha(n))$	$O(1)$

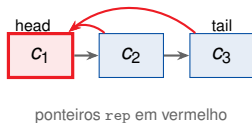
O que aprendemos

- Conjuntos disjuntos modelam relações de equivalência dinâmicas
- Três operações: MAKE-SET, FIND-SET, UNION
- Objetivo: custo total $O(m\alpha(n))$ — praticamente linear
- Duas heurísticas simples permitem atingir esse resultado

Representação por Lista Encadeada

Cada conjunto = uma **lista encadeada**:

- Cada nó tem campo `rep` → representante (`head`)
- A lista mantém ponteiros `head` e `tail`
- `FIND-SET( $x$ )`: retorna $x.rep$ em $O(1)$
- `UNION`: concatena e atualiza todos os `rep`



MAKE-SET(x)

$x.\text{next} \leftarrow \text{NIL}$

$x.\text{rep} \leftarrow x$

novο conjunto : $\{\text{head} = x, \text{tail} = x\}$

Custo: $O(1)$

FIND-SET(x)

return $x.\text{rep}$

Custo: $O(1)$ — acesso direto.

Exemplo: Make-Set(1), Make-Set(2), Make-Set(3)



UNION(x, y) — sem heurística

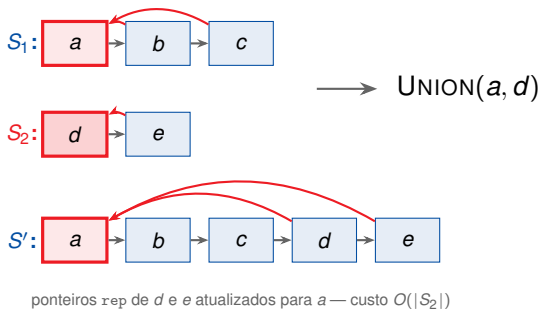
1. $S_x \leftarrow$ conjunto de x ; $S_y \leftarrow$ conjunto de y
2. $S_x.\text{tail}.\text{next} \leftarrow S_y.\text{head}$
3. **Para cada** nó z em S_y : $z.\text{rep} \leftarrow S_x.\text{head}$
4. $S_x.\text{tail} \leftarrow S_y.\text{tail}$

Custo

Passo 3 percorre toda $S_y \Rightarrow O(|S_y|)$.

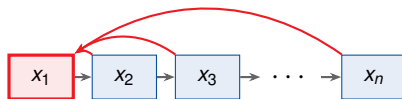
Sem heurística, sequência adversarial leva a $\Theta(n^2)$.

Passo a passo: Union na lista



Sequência adversarial

$\text{UNION}(x_1, x_2), \text{UNION}(x_2, x_3), \dots, \text{UNION}(x_{n-1}, x_n)$
sempre concatenando lista de 1 elemento na lista crescente:
 $\text{custo} = 1 + 2 + \dots + (n - 1) = \Theta(n^2)$.



Regra (CLRS, Seção 19.2)

Anexar sempre a lista **menor** ao final da lista **maior**.
O representante guarda o **tamanho** do conjunto.

Lema chave

Cada nó tem seu ponteiro `rep` atualizado **no máximo** $\lfloor \lg n \rfloor$ vezes.

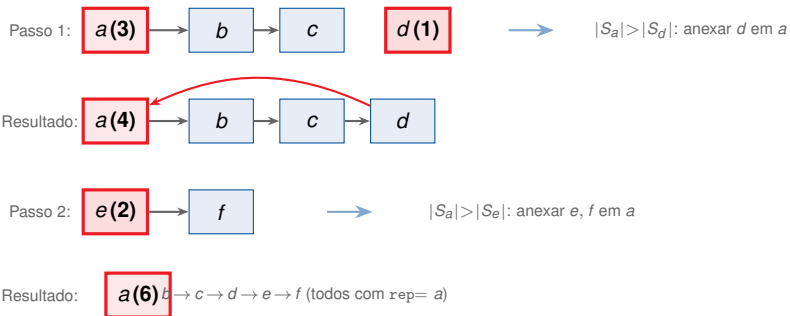
Por quê? A cada atualização, o conjunto ao qual x pertence ao menos **dobra** de tamanho.

Teorema 19.1 (CLRS)

Com m operações totais sobre n elementos:

Custo total: $O(m + n \lg n)$

Passo a passo: Union por Peso



O que aprendemos

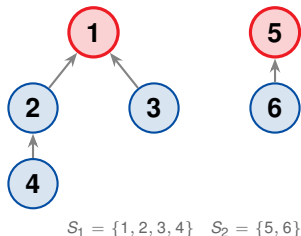
- FIND-SET em $O(1)$; UNION em $O(n)$ sem heurística
- Union por peso: cada nó é atualizado $\leq \lfloor \lg n \rfloor$ vezes
- Custo total: $O(m + n \lg n)$ com union por peso
- Ainda caro para n grande: precisamos evitar percorrer a lista no Union

Representação por Floresta

Ideia: árvores enraizadas

Representação por floresta

- Cada conjunto = uma **árvore enraizada**
- Raiz = representante do conjunto
- Cada nó guarda apenas o ponteiro para o **pai**
- Raiz: $x.\text{parent} = x$



MAKE-SET(x)

$x.\text{parent} \leftarrow x$

$x.\text{rank} \leftarrow 0$ (preparando para Union by Rank)

Custo: $O(1)$

Exemplo: Make-Set(1..6) — seis conjuntos singleton



cada nó é raiz de si mesmo — rank 0

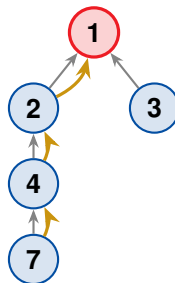
FIND-SET(x) — sem compressão

while $x.\text{parent} \neq x$:

$x \leftarrow x.\text{parent}$

return x

Custo: $O(\text{altura da árvore})$



FIND-SET(7): caminho amarelo até raiz 1

UNION(x, y) — sem rank

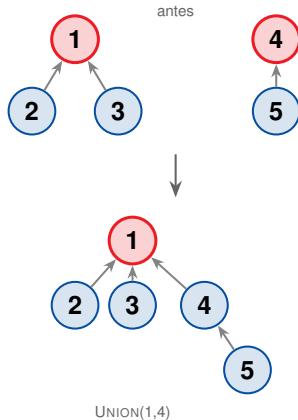
$r_x \leftarrow \text{FIND-SET}(x)$

$r_y \leftarrow \text{FIND-SET}(y)$

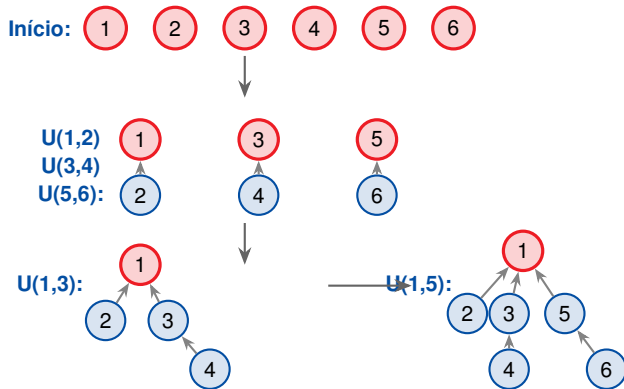
if $r_x \neq r_y$:

$r_y.\text{parent} \leftarrow r_x$

Custo: $O(1)$ após encontrar raízes

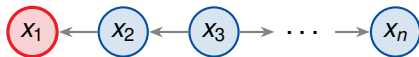


Sequência completa de operações



Sequência adversarial para a floresta ingênua

$\text{UNION}(x_1, x_2), \text{UNION}(x_2, x_3), \dots, \text{UNION}(x_{n-1}, x_n)$ sempre conectando a raiz da árvore maior como filho da menor.



altura = $n - 1 \rightarrow \text{FIND-SET}$ em $O(n)$

	Lista encadeada	Floresta ingênua
MAKE-SET	$O(1)$	$O(1)$
FIND-SET	$O(1)$	$O(n)$
UNION	$O(n)$	$O(1) + \text{FIND-SET}$
Total m ops	$O(mn)$ pior caso	$O(mn)$ pior caso

Ambas são equivalentes no pior caso.
As **heurísticas** fazem a diferença — próximas seções.

```
1 public class UnionFind {
2     private final int[] parent;
3
4     public UnionFind(int n) {
5         parent = new int[n];
6         for (int i = 0; i < n; i++) parent[i] = i; // Make-Set
7     }
8
9     // Find-Set sem compressao -- O(altura)
10    public int find(int x) {
11        while (parent[x] != x) x = parent[x];
12        return x;
13    }
14
15    // Union sem heuristica -- O(1) apos find
16    public void union(int x, int y) {
17        int rx = find(x), ry = find(y);
18        if (rx != ry) parent[ry] = rx;
19    }
20 }
```

O que aprendemos

- Floresta: cada nó guarda apenas ponteiro para o pai
- MAKE-SET e UNION em $O(1)$; FIND-SET em $O(\text{altura})$
- Sem heurísticas, árvore degenera em cadeia $\rightarrow O(n)$ por Find
- Solução: **Union by Rank** controla a altura da árvore

Union by Rank

Problema

Sem critério de escolha, Union pode criar árvores altas.
Find-Set percorre todo o caminho até a raiz.

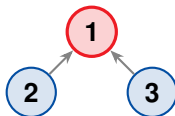
Solução: Union by Rank

Manter um **rank** por nó — limite superior da altura.
No Union, a raiz de menor rank passa a ser filha da de maior rank.

ruim (degenerado)



bom (balanceado)

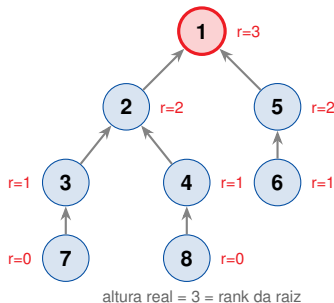


Rank

- $\text{MAKE-SET}(x)$: $x.\text{rank} = 0$
- Se $\text{rank}[r_x] > \text{rank}[r_y]$: $r_y \rightarrow r_x$ (rank inalterado)
- Se ranks iguais: $r_y \rightarrow r_x$ e $r_x.\text{rank} += 1$

Invariante

$\text{rank}(x) \geq \text{altura real da subárvore de } x$.
Árvore de rank $k \Rightarrow \geq 2^k$ nós.



LINK(r_x, r_y)

```
if  $r_x.\text{rank} > r_y.\text{rank}$ :  
     $r_y.\text{parent} \leftarrow r_x$   
elseif  $r_x.\text{rank} < r_y.\text{rank}$ :  
     $r_x.\text{parent} \leftarrow r_y$   
else:  
     $r_y.\text{parent} \leftarrow r_x$   
     $r_x.\text{rank} += 1$ 
```

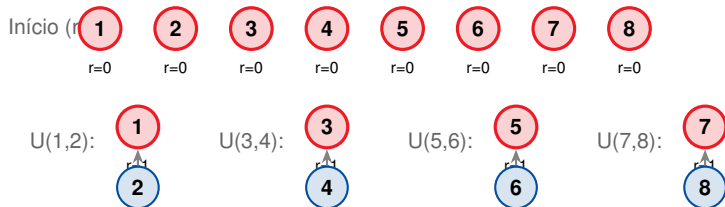
UNION(x, y)

```
LINK(FIND-SET( $x$ ), FIND-SET( $y$ ))
```

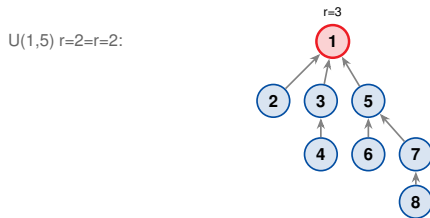
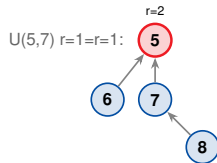
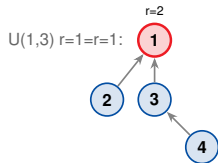
Resultado

Altura máxima da árvore: $\lfloor \lg n \rfloor$
FIND-SET garante $O(\lg n)$ com esta heurística isolada.

Passo a passo: Union by Rank — Parte 1



Passo a passo: Union by Rank — Parte 2



rank cresce apenas quando ranks são iguais

Lema 19.2 (CLRS)

Para qualquer nó x : a subárvore de x tem pelo menos $2^{x.\text{rank}}$ nós.

Corolário

O rank máximo de qualquer nó numa floresta com n elementos é $\lfloor \lg n \rfloor$.
Logo, a altura de qualquer árvore é no máximo $\lfloor \lg n \rfloor$.

Consequência

Com Union by Rank isolado: FIND-SET em $O(\lg n)$.
Para $n = 10^9$: altura ≤ 30 — razoável, mas ainda podemos fazer melhor.

Por que árvore de rank k tem $\geq 2^k$ nós?

Base ($k = 0$): rank 0 $\rightarrow$ subárvore tem $\geq 1 = 2^0$ nó. ✓

Indução ($k > 0$): o rank de r chega a k somente quando LINK une duas raízes de rank $k - 1$.

Por hipótese, cada uma tem $\geq 2^{k-1}$ nós.

A árvore resultante tem $\geq 2^{k-1} + 2^{k-1} = 2^k$ nós. ✓

Propriedade adicional

O número de nós com rank exatamente k é $\leq n/2^k$.

Os ranks são **estritamente crescentes** no caminho de qualquer nó até a raiz.

```
1 public class UnionFindRank {
2     private final int[] parent;
3     private final int[] rank;
4
5     public UnionFindRank(int n) {
6         parent = new int[n];
7         rank   = new int[n];
8         for (int i = 0; i < n; i++) parent[i] = i; // Make-Set
9     }
10
11     public int find(int x) { // Find-Set sem compressao
12         while (parent[x] != x) x = parent[x];
13         return x;
14     }
15
16     public void union(int x, int y) { // Union by Rank
17         int rx = find(x), ry = find(y);
18         if (rx == ry) return;
19         if (rank[rx] < rank[ry]) { int t = rx; rx = ry; ry = t; }
20         parent[ry] = rx; // ry passa a ser filho de rx
21         if (rank[rx] == rank[ry]) rank[rx]++;
22     }
23 }
```

O que aprendemos

- Rank: limite superior da altura do nó
- Union sempre coloca a raiz de menor rank como filha
- Árvore de rank k tem pelo menos 2^k nós
- Altura máxima: $\lfloor \lg n \rfloor \Rightarrow \text{FIND-SET em } O(\lg n)$
- Próximo passo: **Path Compression** para reduzir ainda mais

Path Compression

Custo do Find-Set

Mesmo com Union by Rank, $\text{FIND-SET}(x)$ percorre todo o caminho de x até a raiz.

Chamadas repetidas ao mesmo nó percorrem o mesmo caminho inutilmente.

Ideia

Durante FIND-SET , **comprimir o caminho**: fazer todos os nós visitados apontarem diretamente para a raiz.



FIND-SET recursivo (two-pass)

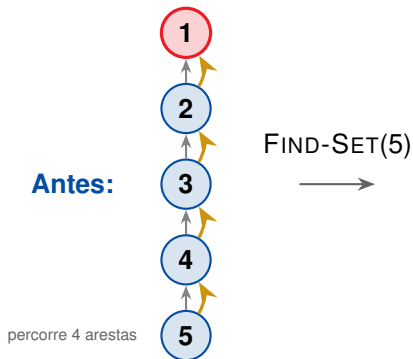
```
if  $x.parent \neq x$ :  
     $x.parent \leftarrow \text{FIND-SET}(x.parent)$   
return  $x.parent$ 
```

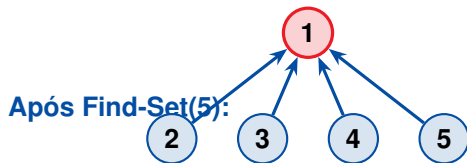
FIND-SET iterativo

Primeiro percurso: encontrar a raiz.
Segundo percurso: atualizar todos os ponteiros para a raiz.

O que muda?

- O valor retornado é idêntico ao sem compressão
- Os ponteiros `parent` são **atualizados como efeito colateral**
- O rank **não é atualizado** (pode ficar maior que a altura real)
- Custo amortizado: $O(\lg n)$ com PC isolado





todos os nós 2,3,4,5 agora apontam diretamente para a raiz 1
próximo Find-Set: $O(1)$ por nó

```
1 public class UnionFindPC {
2     private final int[] parent;
3     private final int[] rank;
4
5     public UnionFindPC(int n) {
6         parent = new int[n];
7         rank   = new int[n];
8         for (int i = 0; i < n; i++) parent[i] = i;
9     }
10
11     // Find-Set com path compression (recursivo)
12     public int find(int x) {
13         if (parent[x] != x)
14             parent[x] = find(parent[x]); // comprime no retorno da recursao
15         return parent[x];
16     }
17
18     public void union(int x, int y) {
19         int rx = find(x), ry = find(y);
20         if (rx == ry) return;
21         if (rank[rx] < rank[ry]) { int t = rx; rx = ry; ry = t; }
22         parent[ry] = rx;
23         if (rank[rx] == rank[ry]) rank[rx]++;
24     }
25 }
```

```
1 // Dois percursos: encontrar raiz, depois comprimir
2 public int find(int x) {
3     // 1. encontrar a raiz
4     int root = x;
5     while (parent[root] != root) root = parent[root];
6
7     // 2. comprimir: todos apontam para root
8     while (parent[x] != root) {
9         int next = parent[x];
10        parent[x] = root;
11        x = next;
12    }
13    return root;
14 }
```

Comparação

- Recursivo: elegante, mas usa pilha de chamadas
- Iterativo: preferível para pilhas rasas e grandes n
- Ambos produzem o mesmo resultado final

O que aprendemos

- PC faz todos os nós no caminho apontarem diretamente para a raiz
- O rank *não* é corrigido — pode ser maior que a altura real
- Custo amortizado com PC isolado: $O(\lg n)$ por operação Find
- PC e Union by Rank *juntos* $\rightarrow O(\alpha(n))$ — próxima seção

Combinação: Rank + Path Compression

Union by Rank

Controla a **estrutura de longo prazo**:
limita a altura durante os Unions.

Path Compression

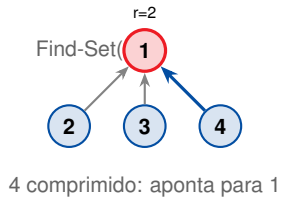
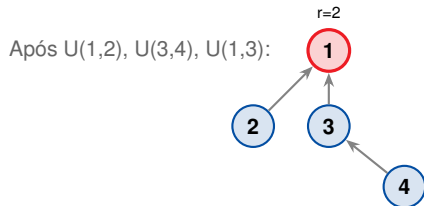
Reduz o custo de **futuras consultas**:
achata a árvore durante os Finds.

Sinergia

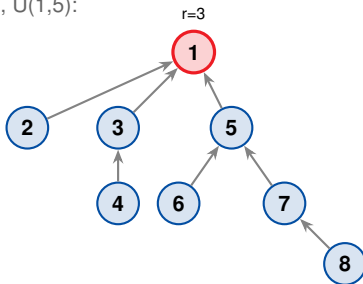
PC achata nós visitados $\rightarrow$ as futuras Finds são baratas.

Rank preserva a propriedade de que as raízes têm rank alto $\rightarrow$ PC só ocorre em caminhos curtos (amortizado).

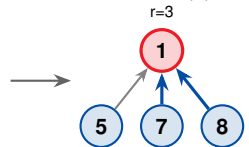
Resultado: $O(\alpha(n))$ amortizado por operação.



Adicionar $U(5,6)$, $U(7,8)$, $U(5,7)$, $U(1,5)$:



Find-Set(8):



7 e 8 comprimidos

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class GenericUnionFind<T> {
5     private final Map<T, T> parent = new HashMap<>();
6     private final Map<T, Integer> rank = new HashMap<>();
7
8     public void makeSet(T x) {
9         parent.put(x, x);
10        rank.put(x, 0);
11    }
12
13    public T find(T x) {           // pre-condicao: makeSet(x) ja chamado
14        if (!parent.get(x).equals(x))
15            parent.put(x, find(parent.get(x))); // path compression
16        return parent.get(x);
17    }
```

```
1 public boolean union(T x, T y) {
2     T rx = find(x), ry = find(y);
3     if (rx.equals(ry)) return false; // ja no mesmo conjunto
4
5     int rankX = rank.get(rx), rankY = rank.get(ry);
6     if (rankX < rankY) { T tmp = rx; rx = ry; ry = tmp; }
7
8     parent.put(ry, rx); // ry passa a ser filho de rx
9     if (rankX == rankY) // ranks iguais: incrementar
10         rank.merge(rx, 1, Integer::sum);
11     return true;
12 }
13
14 public boolean connected(T x, T y) {
15     return find(x).equals(find(y));
16 }
17 }
```

O que aprendemos

- Union by Rank controla a estrutura de longo prazo (altura $\leq \lg n$)
- Path Compression achata o caminho durante cada Find
- Juntos: custo amortizado $O(\alpha(n))$ por operação
- `GenericUnionFind<T>` implementa em Java com `HashMap` e generics

Análise Amortizada

Custo por operação vs. custo total

- Uma única FIND-SET com PC pode ser cara ($O(\lg n)$)
- Mas operações subsequentes sobre os mesmos nós são baratas
- Análise por operação piora o limite real — análise **amortizada** é mais justa

Análise amortizada pelo método do potencial

Atribuímos um **potencial** Φ ao estado da estrutura.

Custo amortizado = custo real + $\Delta\Phi$.

Somando sobre m operações: custo total $\leq \sum$ amortizado + Φ_0 .

Uma função que cresce *incrivelmente* rápido

- $A_0(j) = j + 1$ (sucessor); $A_1(j) = 2j + 1$
- $A_2(j) = 2^{j+1}(j + 1) - 1$; logo $A_2(1) = 7$
- $A_3(1) = A_2(7) = 2047$
- $A_4(1) = A_3(2047)$ — maior que o nº de átomos no universo

Inversa de Ackermann: $\alpha(n)$

$$\alpha(n) = \min\{k : A_k(1) \geq n\}$$

Como $A_k(1)$ explode, $\alpha(n) \leq 4$ para *qualquer* n prático (basta $n \leq A_4(1)$, um número astronômico).

Função de Ackermann $A_k(j)$ (CLRS, Seção 19.4)



Definição (nível $k \geq 0$, argumento $j \geq 1$)

$$A_k(j) = \begin{cases} j + 1 & \text{se } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{se } k \geq 1 \end{cases}$$

onde $A_{k-1}^{(j+1)}$ é A_{k-1} aplicada $j + 1$ vezes (iteração funcional).

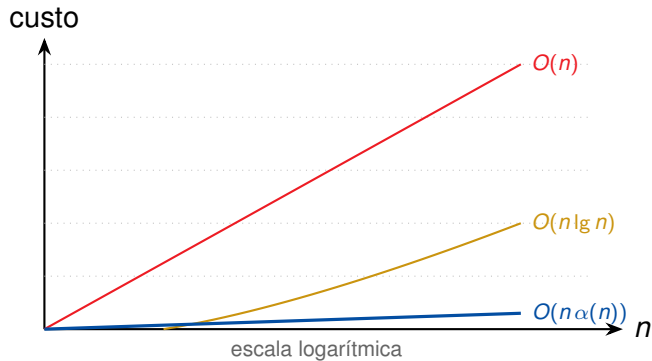
k	$A_k(1)$	fórmula geral
0	2	$A_0(j) = j + 1$
1	3	$A_1(j) = 2j + 1$
2	7	$A_2(j) = 2^{j+1}(j + 1) - 1$
3	2047	$A_3(1) = A_2(7)$
4	$A_4(1) = A_3(2047) \gg 10^{80}$ (astronômico)	

Definição

$\alpha(n) = \min\{k : A_k(1) \geq n\}$ — o menor nível cujo $A_k(1)$ já alcança n .

Faixa de n	$\alpha(n)$
$n \leq 2$	0
$n = 3$	1
$4 \leq n \leq 7$	2
$8 \leq n \leq 2047$	3
$2048 \leq n \leq A_4(1)$	4 (todo n concebível)

Como $A_4(1) \gg 10^{80}$, na prática $\alpha(n) \leq 4$ sempre.



Enunciado

Seja uma sequência de m operações MAKE-SET, UNION e FIND-SET sobre n elementos, usando **union por rank** e **path compression**.

O tempo total no pior caso é:

$$O(m\alpha(n))$$

Significado prático

Como $\alpha(n) \leq 4$ para qualquer n prático, o custo por operação é essencialmente $O(1)$ amortizado.

Por que surge $\alpha(n)$? (intuição)

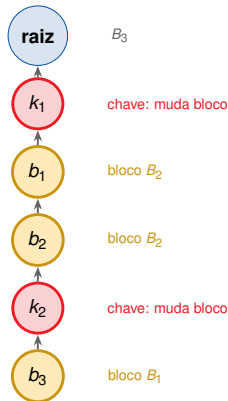
Dois tipos de nó no caminho de Find-Set

Nó de bloco (amarelo): seu pai está no *mesmo bloco de ranks*.

Nó chave (vermelho): seu pai está em *bloco superior*.

A contagem mostra:

- Nós chave por Find: $O(\alpha(n))$
- Custo de nós de bloco: amortizado $O(1)$



2 nós chave $\Rightarrow O(\alpha(n))$ por Find

Definição

$$B_k = \{A_{k-1}(1) + 1, \dots, A_k(1)\},$$
$$B_0 = \{0, \dots, A_0(1)\}$$

Bloco	Ranks
B_0	$\{0, 1, 2\}$
B_1	$\{3\}$
B_2	$\{4, \dots, 7\}$
B_3	$\{8, \dots, 2047\}$
B_4	$\{2048, \dots, A_4(1)\}$

Por que isso importa?

Em qualquer árvore com n nós práticos:

- ranks $\leq \lfloor \lg n \rfloor \leq 30$ (para $n = 10^9$)
- logo só os blocos B_0 – B_3 são usados
- ≤ 4 **blocos** $\Rightarrow O(\alpha(n))$ saltos por Find-Set

Esta é a origem do $\alpha(n)$.

Custo total de m operações

MAKE-SET: n operações, $O(1)$ cada $\rightarrow$ total $O(n)$

UNION: $\leq n - 1$ operações, $O(1)$ cada $\rightarrow$ total $O(n)$

FIND-SET: Nós chave por chamada: $O(\alpha(n))$;
nós de bloco: custo amortizável ao potencial $\rightarrow O(1)$ total por op.

Resultado

Custo total = $O(m\alpha(n))$. No modelo de *pointer machine* este limite é justo: Fredman–Saks (1989) provam $\Omega(m\alpha(n))$ no pior caso.

Implementação	Por op. (amort.)	m operações
Lista ingênua	$O(n)$	$O(mn)$
Lista + union por peso	$O(1)$ Find, amort. $O(\lg n)$ Union	$O(m + n \lg n)$
Floresta + rank	$O(\lg n)$	$O(m \lg n)$
Floresta + PC	$O(\lg n)$	$O(m \lg n)$
Floresta + rank + PC	$O(\alpha(n))$	$O(m \alpha(n))$

$\alpha(n) \leq 4$ para qualquer n prático

O que aprendemos

- Análise amortizada revela o custo médio real por operação
- Função de Ackermann cresce mais rápido que qualquer função primitiva recursiva
- $\alpha(n) \leq 4$ para qualquer n concebível $\rightarrow$ prática é $O(1)$ amortizado
- Teorema 19.14 (CLRS): $O(m \alpha(n))$ no pior caso (justo no modelo de *pointer machine*)

Implementação em Java Moderno

Para Union-Find genérico

- **Generics** (<T>): suportar qualquer tipo
- **HashMap**: mapeamento elemento → pai
- **Records**: imutabilidade para metadados
- **var**: inferência de tipo local

Para uso em grafos

- **Stream API**: processar arestas
- **Comparator**: ordenar por peso
- **instanceof pattern**: verificar tipos
- **sealed interfaces**: modelar tipos fechados

```
1 public interface DisjointSet<T> {
2     /** Cria um conjunto unitario {x}. */
3     void makeSet(T x);
4
5     /** Retorna o representante do conjunto de x. */
6     T find(T x);
7
8     /**
9      * Une os conjuntos de x e y.
10     * @return true se estavam em conjuntos diferentes
11     */
12    boolean union(T x, T y);
13
14    /** Verifica se x e y estao no mesmo conjunto. */
15    default boolean connected(T x, T y) {
16        return find(x).equals(find(y));
17    }
18 }
```

```
1 public final class ArrayUnionFind {
2     private final int[] parent;
3     private final int[] rank;
4
5     public ArrayUnionFind(int n) {
6         parent = new int[n]; rank = new int[n];
7         for (int i = 0; i < n; i++) parent[i] = i;
8     }
9
10    public int find(int x) {
11        if (parent[x] != x) parent[x] = find(parent[x]);
12        return parent[x];
13    }
14
15    public boolean union(int x, int y) {
16        var rx = find(x); var ry = find(y);
17        if (rx == ry) return false;
18        if (rank[rx] < rank[ry]) { var t = rx; rx = ry; ry = t; }
19        parent[ry] = rx;
20        if (rank[rx] == rank[ry]) rank[rx]++;
21        return true;
22    }
23
24    public boolean connected(int x, int y) { return find(x) == find(y); }
25 }
```

```
1 // Record: imutavel, equals/hashCode/toString automaticos (Java 16+)
2 public record Edge<V>(V from, V to, double weight)
3     implements Comparable<Edge<V>> {
4
5     @Override
6     public int compareTo(Edge<V> other) {
7         return Double.compare(this.weight, other.weight);
8     }
9 }
10
11 // Uso com var e generics:
12 var edges = List.of(
13     new Edge<>("A", "B", 3.0),
14     new Edge<>("B", "C", 1.5),
15     new Edge<>("A", "C", 4.2)
16 );
```

```
1 public static <V> List<Edge<V>> kruskal(  
2     Collection<V> vertices,  
3     Collection<Edge<V>> edges) {  
4  
5     var uf = new GenericUnionFind<V>();  
6     vertices.forEach(uf::makeSet);  
7  
8     return edges.stream()  
9         .sorted() // ordena por peso (Comparable)  
10        // efeito colateral seguro apenas em stream sequencial (nao use .parallel())  
11        .filter(e -> uf.union(e.from(), e.to())) // false = ciclo  
12        .collect(Collectors.toList());  
13 }  
14  
15 // Uso:  
16 var mst = kruskal(graph.vertices(), graph.edges());  
17 mst.forEach(e -> System.out.printf("%s--%s: %.1f%n",  
18     e.from(), e.to(), e.weight()));
```



```
1 public static <V> boolean hasCycle(  
2     Collection<V> vertices,  
3     Collection<Edge<V>> edges) {  
4  
5     var uf = new GenericUnionFind<V>();  
6     vertices.forEach(uf::makeSet);  
7  
8     for (var edge : edges) {  
9         // union retorna false se ja estao conectados = ciclo  
10        if (!uf.union(edge.from(), edge.to())) {  
11            return true;  
12        }  
13    }  
14    return false;  
15 }
```

Custo

$O(m\alpha(n))$ para m arestas e n vértices.

```
1 import org.junit.jupiter.api.*;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 class UnionFindTest {
5     private ArrayUnionFind uf;
6
7     @BeforeEach void setUp() { uf = new ArrayUnionFind(10); }
8
9     @Test void singletons_are_not_connected() {
10         assertFalse(uf.connected(0, 1));
11         assertFalse(uf.connected(3, 7));
12     }
13
14     @Test void union_connects_elements() {
15         uf.union(0, 1); uf.union(1, 2);
16         assertTrue(uf.connected(0, 2)); // transitivo
17         assertFalse(uf.connected(0, 3));
18     }
19 }
```

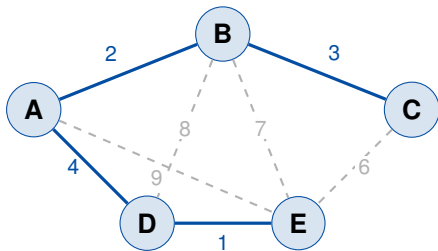
```
1  @Test void find_is_idempotent() {
2      uf.union(4, 5);
3      int r1 = uf.find(4);
4      int r2 = uf.find(4);
5      assertEquals(r1, r2);
6  }
7
8  @Test void union_returns_false_if_already_connected() {
9      uf.union(6, 7);
10     boolean first = uf.union(6, 7); // mesmos
11     boolean second = uf.union(7, 6); // ordem invertida
12     assertFalse(first);
13     assertFalse(second);
14 }
15
16 @Test void kruskal_produces_mst_with_n_minus_1_edges() {
17     var vertices = List.of(0, 1, 2, 3);
18     var edges = List.of(new Edge<>(0,1,1.0), new Edge<>(1,2,2.0),
19                        new Edge<>(2,3,3.0), new Edge<>(0,3,9.0));
20     var mst = Kruskal.kruskal(vertices, edges);
21     assertEquals(3, mst.size());
22 }
23 }
```

O que aprendemos

- Interface `DisjointSet<T>`: contrato limpo com default
- `GenericUnionFind<T>`: `HashMap` para tipos arbitrários
- `ArrayUnionFind`: arrays para performance máxima com inteiros
- `Record Edge<V>`: imutável, comparável, sem boilerplate
- Kruskal com Stream API: elegante e conciso
- JUnit 5: testes que documentam o comportamento esperado

Aplicações

Kruskal: Árvore Geradora Mínima



MST: $\{D-E, A-B, B-C, A-D\}$, peso total = 10

Algoritmo de Kruskal

1. Ordenar arestas por peso crescente
2. Para cada aresta (u, v) em ordem:
Se $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$:
adicionar à MST
 $\text{UNION}(u, v)$
3. Parar quando MST tem $n - 1$ arestas

Execução (exemplo anterior)

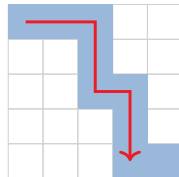
$D-E$ (1): $\text{UNION}(D, E)$
 $A-B$ (2): $\text{UNION}(A, B)$
 $B-C$ (3): $\text{UNION}(B, C)$
 $A-D$ (4): $\text{UNION}(A, D)$
 $C-E$ (6): ciclo — ignora
 $B-E$ (7): ciclo — ignora

Problema

Grade $n \times n$ de sítios.

Cada sítio é aberto com probabilidade p .

O sistema **percola** se existe caminho de sítios abertos do topo à base.



sistema percola

Solução com Union-Find

Nós virtuais: *top* e *bottom*.

Ao abrir sítio (i, j) : unir com vizinhos abertos.

Percola $\iff$ FIND-SET(*top*) =
FIND-SET(*bottom*).

Compiladores

- Unificação de tipos em Prolog/ML
- Análise de equivalência de registradores
- Eliminação de variáveis duplicadas (SSA)

Processamento de imagens

- Segmentação de objetos em imagens binárias
- Flood fill com componentes conexas

Redes e sistemas

- Detecção de clusters em redes P2P
- Controle de congestionamento (grupos de enlace)
- Gerência de memória: blocos livres contíguos

Jogos e simulações

- Geração de labirintos (Kruskal)
- Física: simulação de materiais granulares

Técnica	Make	Find	Union	Total
Lista ingênua	$O(1)$	$O(1)$	$O(n)$	$O(mn)$
Lista + peso	$O(1)$	$O(1)$	$O(\lg n)$	$O(m+n \lg n)$
Floresta ingênua	$O(1)$	$O(n)$	$O(1)$	$O(mn)$
+ rank	$O(1)$	$O(\lg n)$	$O(1)$	$O(m \lg n)$
+ PC	$O(1)$	$O(\lg n)$	$O(1)$	$O(m \lg n)$
+ rank+PC	$O(1)$	$O(\alpha)$	$O(1)$	$O(m\alpha(n))$

$\alpha = \alpha(n) \leq 4$ para qualquer n prático

Jornada desta aula

1. **Problema:** conectividade dinâmica — relação de equivalência
2. **Lista encadeada:** Find $O(1)$, Union $O(n) \rightarrow$ melhor: $O(m+n \lg n)$
3. **Floresta:** Union $O(1)$, Find $O(\text{altura})$
4. **Union by Rank:** controla altura $\rightarrow O(\lg n)$ por Find
5. **Path Compression:** achata o caminho $\rightarrow$ amortiza futuras Finds
6. **Juntos:** $O(\alpha(n)) \approx O(4)$ — praticamente $O(1)$
7. **Java:** GenericUnionFind<T>, Record, Stream, JUnit 5

- **CLRS 4^a ed.**, Capítulo 19 — *Data Structures for Disjoint Sets*:
Seções 19.1–19.4: definições, listas, floresta, análise amortizada completa
- **Tarjan, R.E. (1975)**: *Efficiency of a Good But Not Linear Set Union Algorithm*
— artigo original da análise com $\alpha(n)$
- **Sedgewick & Wayne** — *Algorithms*, 4^a ed., Seção 1.5: abordagem prática com foco em Java
- **Skiena** — *The Algorithm Design Manual*, Cap. 8: Union-Find no contexto de grafos

1. (CLRS 19.1-1) Se `FIND-SET( $x$ )` for chamado em um nó x cujo pai não é a raiz, o que acontece ao rank de x após a compressão?
2. Implemente `nComponents(int n, int[][] edges)` que conta o número de componentes conexas de um grafo não-dirigido.
3. Mostre que a heurística de união por peso na lista encadeada garante $\lfloor \lg n \rfloor$ atualizações de ponteiro no máximo por elemento.
4. Implemente percolação Monte Carlo: estime a probabilidade de percolação para $p \in [0, 1]$ numa grade 20×20 com 10 000 experimentos.
5. (Desafio) Implemente uma versão thread-safe de `ArrayUnionFind` usando `AtomicIntegerArray`.

-  Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 4 edition, 2022.
-  Robert Sedgewick and Kevin Wayne.
Algorithms.
Addison-Wesley Professional, 4 edition, 2011.
-  Robert Endre Tarjan.
Efficiency of a good but not linear set union algorithm.
Journal of the ACM, 22(2):215–225, 1975.



Robert Endre Tarjan and Jan van Leeuwen.
Worst-case analysis of set union algorithms.
Journal of the ACM, 31(2):245–281, 1984.